



Technical Report - Commands & Configurations

AI-Powered Intrusion Detection System for Cloud Environments

KRID Aboubakr

Master of Artificial Intelligence and Cybersecurity

Supervised by Prof. Fakhri Yousef

May 2026

Contents

1	Host Machine Verification	3
1.1	Hardware Virtualisation Check	3
1.2	Disk Layout Inspection	3
2	Storage Setup - /mnt/lab Partition	4
2.1	Identify Unallocated Space	4
2.2	Create the New ext4 Partition	4
2.3	Create Directory Structure	4
2.4	Verify Partition	4
3	KVM Installation and Configuration	5
3.1	Install KVM Tools	5
3.2	Configure KVM Storage Pool	5
4	Virtual Network Setup	6
4.1	Download ISO Files	6
4.2	Create the ids-lab Isolated Network	6
5	VM1 - Ubuntu Server 22.04	8
5.1	Create the VM	8
5.2	VM1 Installation Settings	8
5.3	Network Interfaces After Installation	8
5.4	SSH Access from Host	8
5.5	Internet Connectivity Test	9
5.6	Install Required Software on VM1	9
5.6.1	System Update	9
5.6.2	Install Docker Inside VM1	9
5.6.3	Install Network and Java Tools	9
5.7	Verification of VM1 Tools	10
6	VM2 - Kali Linux 2026.1	11
6.1	Create the VM	11
6.2	VM2 Installation Settings	11
6.3	Fix Network Configuration on VM2	11
6.3.1	Activate eth0 (ids-lab)	11
6.3.2	Create Permanent Profile for eth1 (Internet)	12
6.3.3	Prevent Routing Conflict	12
6.3.4	Activate Both Interfaces	12

6.4	Verified Network State on VM2	12
7	Connectivity Verification	13
7.1	VM1 to VM2 Ping Test	13
7.2	Internet Test from Both VMs	13

1. Host Machine Verification

1.1 Hardware Virtualisation Check

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

```
32
```

Result: 32 vmx entries confirmed. KVM hardware virtualisation is available.

1.2 Disk Layout Inspection

```
lsblk
```

NAME	SIZE	FSTYPE	MOUNTPOINT	LABEL
nvme0n1	476.9G			
+nvme0n1p1	300M	vfat		SYSTEM
+nvme0n1p2	128M			
+nvme0n1p3	209.9G	ntfs		Windows
+nvme0n1p4	64.7G	ntfs	/media/akrid/D	
+nvme0n1p5	900M	ntfs		WinRE tools
+nvme0n1p6	20.8G	ntfs		BIOS_RVY
+nvme0n1p7	1000M	vfat	/boot/efi	
+nvme0n1p8	99G	ext4	/	

Note

The Windows D: partition (nvme0n1p4) had already been shrunk from 144.9 GB to 64.7 GB using Windows Disk Management, freeing approximately 80 GB of unallocated space visible to **parted** but not listed by **lsblk**.

2. Storage Setup - /mnt/lab Partition

2.1 Identify Unallocated Space

```
sudo parted /dev/nvme0n1 print free
```

2.2 Create the New ext4 Partition

```
sudo parted /dev/nvme0n1
(parted) mkpart primary ext4 398GB 477GB
(parted) quit
```

```
sudo partprobe /dev/nvme0n1
sudo mkfs.ext4 /dev/nvme0n1p9 -L "lab-data"
sudo mkdir /mnt/lab
echo "LABEL=lab-data /mnt/lab ext4 defaults 0 2" | sudo tee -a /
etc/fstab
sudo mount -a
```

2.3 Create Directory Structure

```
sudo mkdir -p /mnt/lab/docker
sudo mkdir -p /mnt/lab/vms
sudo mkdir -p /mnt/lab/datasets
```

2.4 Verify Partition

```
df -h /mnt/lab
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/nvme0n1p9	79G	316K	75G	1%	/mnt/lab

3. KVM Installation and Configuration

3.1 Install KVM Tools

```
sudo apt install qemu-kvm libvirt-daemon-system virt-manager  
bridge-utils  
sudo usermod -aG libvirt $USER  
sudo usermod -aG kvm $USER  
newgrp libvirt
```

3.2 Configure KVM Storage Pool

```
sudo virsh pool-destroy default  
sudo virsh pool-undefine default  
sudo virsh pool-define-as default dir --target /mnt/lab/vms  
sudo virsh pool-autostart default  
sudo virsh pool-start default
```

```
sudo virsh pool-list --all
```

Name	State	Autostart
default	active	yes

4. Virtual Network Setup

4.1 Download ISO Files

```
wget -P /mnt/lab/ \
https://releases.ubuntu.com/22.04/ubuntu-22.04.5-live-server-
amd64.iso

wget -P /mnt/lab/ \
https://cdimage.kali.org/current/kali-linux-2026.1-installer-
amd64.iso
```

```
ls -lh /mnt/lab/*.iso
```

```
-rw-r--r-- 1 root root 4.5G Mar 20 05:20 kali-linux-2026.1-
installer-amd64.iso
-rw-r--r-- 1 root root 2.0G Sep 11 2024 ubuntu-22.04.5-live-
server-amd64.iso
```

4.2 Create the ids-lab Isolated Network

```
sudo virsh net-define /dev/stdin <<EOF
<network>
  <name>ids-lab</name>
  <bridge name="virbr-ids"/>
  <ip address="192.168.100.1" netmask="255.255.255.0">
    <dhcp>
      <range start="192.168.100.10" end="192.168.100.50"/>
    </dhcp>
  </ip>
</network>
EOF

sudo virsh net-autostart ids-lab
sudo virsh net-start ids-lab
```

```
sudo virsh net-list --all
```

Name	State	Autostart	Persistent
default	active	yes	yes

ids-lab	active	yes	yes
---------	--------	-----	-----

5. VM1 - Ubuntu Server 22.04

5.1 Create the VM

```
sudo virt-install \
  --name vm1-ubuntu-ids \
  --ram 5120 \
  --vcpus 4 \
  --disk path=/mnt/lab/vms/vm1-ubuntu-ids.qcow2,size=35,format=
    qcow2 \
  --cdrom /mnt/lab/ubuntu-22.04.5-live-server-amd64.iso \
  --network network=ids-lab,model=virtio \
  --network network=default,model=virtio \
  --os-variant ubuntu22.04 \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole \
  --boot cdrom,hd
```

5.2 VM1 Installation Settings

Setting	Value
Hostname	ids-server
Username	akrid
OpenSSH	Enabled
Snaps	None selected
Disk	Use entire disk (35 GB)

5.3 Network Interfaces After Installation

The installer automatically configured both interfaces via DHCP:

```
enp1s0  -->  192.168.100.46/24    (ids-lab network)
enp2s0  -->  192.168.122.206/24  (default NAT / internet)
```

5.4 SSH Access from Host

```
ssh akrid@192.168.100.46
```

```
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-176-generic
x86_64)
System load:  0.0                Processes:            148
Usage of /:   21.3% of 34.15GB   Users logged in:     0
Memory usage: 4%                IPv4 address for enp2s0:
192.168.122.206
```

5.5 Internet Connectivity Test

```
ping -c 3 8.8.8.8
```

```
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=48.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=37.0 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=32.6 ms
```

5.6 Install Required Software on VM1

5.6.1 System Update

```
sudo apt update && sudo apt upgrade -y
```

5.6.2 Install Docker Inside VM1

```
sudo apt install -y ca-certificates curl gnupg lsb-release

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker
  -compose-plugin

sudo usermod -aG docker akrid
newgrp docker
```

5.6.3 Install Network and Java Tools

```
sudo apt install -y tcpdump tshark net-tools openjdk-17-jre \
  python3 python3-pip git
```

5.7 Verification of VM1 Tools

```
docker --version
docker compose version
java -version
tcpdump --version
python3 --version
```

```
Docker version 29.4.1, build 055a478
Docker Compose version v5.1.3
openjdk version "17.0.18" 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-122.04.1)
tcpdump version 4.99.1
Python 3.10.12
```

```
df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/vda2	35G	8.5G	24G	27%	/

6. VM2 - Kali Linux 2026.1

6.1 Create the VM

```
sudo virt-install \
  --name vm2-kali-attacker \
  --ram 3072 \
  --vcpus 2 \
  --disk path=/mnt/lab/vms/vm2-kali-attacker.qcow2,size=30,
    format=qcow2 \
  --cdrom /mnt/lab/kali-linux-2026.1-installer-amd64.iso \
  --network network=ids-lab,model=virtio \
  --network network=default,model=virtio \
  --os-variant debian12 \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole \
  --boot cdrom,hd
```

6.2 VM2 Installation Settings

Setting	Value
Hostname	kali-attacker
Username	akrid
Desktop	Xfce (lightest option, 200 MB idle)
Software bundle	Top 10 tools
Primary NIC	eth0 (ids-lab network)

6.3 Fix Network Configuration on VM2

After installation, eth0 (ids-lab) had no IPv4 address and eth1 (internet) was not activated. Both were fixed using NetworkManager.

6.3.1 Activate eth0 (ids-lab)

```
sudo nmcli device connect eth0
```

```
Device 'eth0' successfully activated with '...'
```

6.3.2 Create Permanent Profile for eth1 (Internet)

```
sudo nmcli connection add \  
  type ethernet \  
  ifname eth1 \  
  con-name "eth1-internet" \  
  ipv4.method auto \  
  ipv4.never-default no \  
  connection.autoconnect yes
```

6.3.3 Prevent Routing Conflict

Set eth0 to never install a default gateway (internal use only):

```
sudo nmcli connection modify "Wired connection 1" ipv4.never-  
  default yes
```

6.3.4 Activate Both Interfaces

```
sudo nmcli connection up "Wired connection 1"  
sudo nmcli connection up "eth1-internet"
```

6.4 Verified Network State on VM2

```
eth0  -->  192.168.100.28/24    (ids-lab, internal only)  
eth1  -->  192.168.122.219/24  (internet via NAT)
```

7. Connectivity Verification

7.1 VM1 to VM2 Ping Test

Run from VM1 (`ssh akrid@192.168.100.46`):

```
ping -c 4 192.168.100.28
```

```
PING 192.168.100.28 (192.168.100.28) 56(84) bytes of data.  
64 bytes from 192.168.100.28: icmp_seq=1 ttl=64 time=1.11 ms  
64 bytes from 192.168.100.28: icmp_seq=2 ttl=64 time=1.25 ms  
64 bytes from 192.168.100.28: icmp_seq=3 ttl=64 time=1.21 ms  
64 bytes from 192.168.100.28: icmp_seq=4 ttl=64 time=1.00 ms  
--- 192.168.100.28 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 1.003/1.142/1.247/0.094 ms
```

7.2 Internet Test from Both VMs

```
# From VM1  
ping -c 3 8.8.8.8      # Success: ~40ms RTT  
  
# From VM2  
ping -c 3 8.8.8.8      # Success after eth1 configuration
```